PROJECT SYNOPSIS / REPORT

PARALLEL WILDFIRE SPREAD SIMULATION: A SERIAL, OPENMP AND CUDA COMPARATIVE STUDY

PROJECT REPORT

OF HPC INTERNSHIP

BACHELOR OF TECHNOLOGY

Computer Science Engineering / Information Technology

SUBMITTED BY

[Student Name] ([Roll No.])

July 2026

SUPERVISOR

[Supervisor Name]

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING AND INFORMATION TECHNOLOGY

[UNIVERSITY / INSTITUTE NAME]

INDEX

Section Page

Abstract 3

1. Introduction 4

1.1 Background and Motivation 4

1.2 Problem Statement 4

1.3 Scope and Contributions 4

1.4 Organization of the Report 5

2. Objectives 7

3. Application of the Project 8

4. Literature Review 9

4.1 Taxonomy of Wildfire Spread Models 9

4.2 Comparative Analysis of Existing Methods 9

4.3 Research Gap and Positioning of This Work 10

5. Methodology / Planning of Work 11

5.1 Grid Representation and State Model 11

5.2 Per-Step State Transition Rules 11

5.3 Serial Implementation 12

5.4 OpenMP Parallel Implementation 13

5.5 CUDA GPU Implementation 14

5.6 Benchmarking Plan 14

5.7 Dataset Ingestion and Validation Pipeline 15

6. Integration of Ramanujan (RU) HPC Cluster in this Work 17

7. Results and Experiments 19

7.1 Correctness Verification 19

7.2 Runtime Scaling and Speedup 19

7.3 Validation Against Real Wildfire Data (IoU Analysis) 21

7.4 Recommended Secondary Metric: Tolerance-Adjusted IoU 24

8. Conclusion and Future Scope 26

8.1 Conclusion 26

8.2 Limitations of the Present Study 26

8.3 Future Scope 27

9. Bibliography / References 29

List of Tables

Table 1 Comparison of representative wildfire-spread modelling methods 10

Table 2 Objectives mapped to deliverables and success criteria 7

Table 3 Application domains and the role of the simulator 8

Table 4 Cell state encoding and per-cell attributes 11
Table 5 Simulation parameters exposed through SimParams 15
Table 6 Benchmark configuration matrix 15
Table 7 Cross-implementation correctness verification 19
Table 8 Measured wall-clock runtime vs. grid size 20
Table 9 Speedup and parallel efficiency derived from Table 8 20
Table 10 Reference (illustrative) runtime figures from project documentation 21
Table 11 Single-sample IoU diagnostic 22
Table 12 Parameter sweep on steps and base_rate 22
Table 13 Per-sample IoU against the persistence baseline 23
Table 14 Raw vs. tolerance-adjusted (dilated) IoU 24
Table 15 Summary of limitations and mitigations 27

List of Figures

Figure 1 End-to-end project workflow 6
Figure 2 Taxonomy of wildfire spread modelling approaches 9
Figure 3 Cell state transition diagram 11
Figure 4 Moore neighbourhood and wind-biased ignition weighting 12
Figure 5 Ping-pong (double-buffered) update scheme 13
Figure 6 CUDA thread-to-cell mapping 14
Figure 7 Data ingestion pipeline from TFRecord to simulator input 16
Figure 8 Build and execution workflow on the Ramanujan cluster 17
Figure 9 Measured runtime vs. grid size (log scale) 19
Figure 10 Speedup relative to the serial baseline 20
Figure 11 Mask sparsity and simulator over-prediction per sample 23
Figure 12 Effect of step count on predicted burn extent and IoU 23
Figure 13 Raw vs. tolerance-adjusted IoU against a naive baseline 25

ABSTRACT

Wildfires are among the most destructive natural hazards, and computational simulation of their spread is essential for disaster preparedness, resource allocation, and land-management planning. Operational fire-behaviour services are required to produce forecasts within minutes of an ignition report, which makes the raw computational throughput of the underlying spread model a first-class engineering concern rather than an implementation detail. This project implements a two-dimensional cellular-automaton (CA) wildfire spread model in three computational paradigms — a serial CPU baseline, a shared-memory parallel version using OpenMP, and a massively parallel GPU version using CUDA — all sharing the exact same underlying physics and a deterministic hash-based random number generator, so that results can be verified for agreement across implementations independently of thread scheduling or execution order.

Each grid cell carries a fuel load and a moisture value; at every time step, unburned cells adjacent to burning cells are subjected to a wind- and distance-weighted ignition probability, and burning cells remain lit for a fixed burn duration before transitioning to a permanently burnt state. The update is double-buffered, so that every cell reads only the previous step's grid and writes only to its own output slot, rendering the per-step update embarrassingly parallel and free of write conflicts. The project benchmarks runtime scaling across grid sizes from 200×200 to 4000×4000 and across OpenMP thread counts, and additionally reports a validation study in which the simulator's predicted fire growth is compared against reference fire-progression masks derived from the Google Research "Next Day Wildfire Spread" dataset using the Intersection-over-Union (IoU) metric, benchmarked against a naive persistence baseline.

Results confirm the expected performance ordering across the measured grid-size range: the serial implementation is slowest, OpenMP is intermediate, and CUDA becomes the fastest implementation once the grid is large enough to amortize kernel-launch and host-device transfer overhead, reaching a peak measured speedup of approximately 3.7× over the serial baseline at 2000×2000. A crossover point is observed between 200×200 and 500×500, below which the GPU implementation is in fact slower than the CPU baseline — an instructive and frequently under-reported characteristic of fine-grained GPU workloads. Final burning and burnt cell counts match exactly across all three implementations, confirming correctness of the CUDA port before its timing figures are trusted.

The validation study finds that on this sparse-mask dataset even a do-nothing persistence baseline scores a near-zero mean IoU of 0.018, and the simulator attains 0.007 — comparable to, rather than dramatically worse than, that baseline. This outcome is framed honestly as a dataset and metric limitation rather than as a simulator failure. Applying a tolerance-adjusted (dilated) IoU with a one-pixel structuring element raises both scores to 0.039 and 0.023 respectively, preserving their relative ordering while roughly tripling the measured signal, and is therefore recommended as a fairer secondary metric for sparse fire masks. The report concludes by outlining future extensions including elevation-aware spread, shared-memory CUDA optimization, and GPU-parallel Monte-Carlo burn-probability heatmaps.

Keywords: High-Performance Computing; Cellular Automata; Wildfire Spread Simulation; CUDA; OpenMP; GPU Acceleration; Parallel Efficiency; Intersection-over-Union; Sparse Segmentation; Ramanujan HPC Cluster.

1. INTRODUCTION

1.1 Background and Motivation

Wildfire behaviour is governed by a complex interaction of fuel availability, moisture content, terrain, and wind. The resulting dynamics are strongly non-linear: a modest change in wind direction or in fuel moisture can convert a slow, containable ground fire into a rapidly advancing crown fire within a single burning period. Because the outcome depends on the joint evolution of many spatially distributed quantities, closed-form analytical prediction is intractable in all but the most idealized cases, and practitioners rely instead on numerical simulation.

Among the available numerical formulations, the cellular automaton (CA) occupies a particularly attractive position. A CA discretizes the landscape into a regular grid of cells, assigns each cell a small set of discrete states, and evolves the whole grid forward in time by applying an identical local transition rule to every cell simultaneously. The rule depends only on a cell's own attributes and on the states of its immediate neighbours. This locality is what makes the CA formulation both conceptually simple and computationally attractive: because no cell requires global information, the per-step update of every cell in the grid is mutually independent and can, in principle, be performed concurrently.

That property makes CA fire models an unusually good vehicle for studying the progression from serial to shared-memory to massively-parallel computation, which is the core theme of this High-Performance Computing (HPC) internship. The same algorithm can be expressed, without altering its semantics, as a pair of nested loops on a single CPU thread, as a work-shared loop across CPU cores under OpenMP, or as a launch of one GPU thread per grid cell under CUDA. The differences between these three expressions are purely mechanical, which means any observed difference in output is unambiguously a defect rather than a legitimate consequence of the parallelization strategy — a property this project deliberately exploits for verification.

1.2 Problem Statement

Naively parallelizing a stochastic simulation destroys reproducibility. If each thread draws from a shared or thread-local pseudo-random stream, the sequence of random values consumed by any given cell depends on how the runtime happened to schedule the threads on that particular execution. Two runs of the same OpenMP binary with the same inputs will then produce different fire perimeters, and a CUDA port cannot be compared against a serial reference at all, because there is no expectation that they should agree. The engineer is left unable to distinguish a genuine porting bug from ordinary stochastic variation.

This project addresses that problem directly by replacing the stateful random number generator with a deterministic hash function of the cell coordinates, the step index, and a global seed. Because the hash is a pure function of position and time rather than of consumption order, every implementation draws precisely the same random value for a given cell at a given step, irrespective of thread scheduling. Bit-level agreement between the serial, OpenMP, and CUDA outputs therefore becomes a meaningful and achievable correctness criterion, and the project can proceed to performance measurement on a verified basis.

1.3 Scope and Contributions

This project implements a two-dimensional grid in which each cell holds a fuel value, a moisture value, and a state drawn from {UNBURNED, BURNING, BURNT}. At every simulation step, unburned cells examine their eight neighbours; if a neighbour is burning, an ignition probability is computed from local fuel, moisture, wind direction and strength, and the relative offset to that neighbour. The three implementations — serial.cpp, openmp_sim.cpp, and cuda_sim.cu — share identical physics and RNG logic defined in a common header, common.h, and differ only in how the per-cell update loop is executed. The specific contributions are as follows.

- A shared-physics, three-paradigm implementation of a probabilistic CA wildfire model in which the ignition function and the random number generator are defined exactly once, so that any future refinement to the physics is inherited automatically and identically by all three execution backends.

- A deterministic hash-based random number generator that makes exact cross-implementation output comparison possible, and the demonstration that all three implementations produce identical burning and burnt cell counts under identical inputs.
- A benchmark study of wall-clock runtime across four grid sizes spanning a 400-fold range in cell count, together with derived speedup and parallel-efficiency figures and an analysis of the GPU crossover point below which acceleration is counter-productive.
- An external validation study against real satellite-derived fire masks using IoU, deliberately accompanied by a naive persistence baseline so that the resulting scores can be interpreted against the intrinsic difficulty of the task rather than against an unstated assumption that a high IoU should be attainable.
- A diagnosis of raw IoU as an inappropriate primary metric at single-digit mask sizes, and the empirical demonstration that a tolerance-adjusted (dilated) IoU recovers roughly three times the measured signal while preserving the ordering between the simulator and the baseline.

1.4 Organization of the Report

Section 2 states the objectives of the work and maps each to a concrete deliverable and success criterion. Section 3 surveys the application domains in which such a simulator is used. Section 4 reviews the wildfire modelling literature and positions the present approach within it. Section 5 sets out the methodology in detail, covering the state model, the transition rules, each of the three implementations, the benchmarking plan, and the dataset ingestion pipeline. Section 6 describes how the Ramanujan HPC cluster was integrated into the build and execution workflow. Section 7 presents the correctness, performance, and validation results. Section 8 concludes and outlines future scope, and Section 9 lists the references consulted.



Figure 1: End-to-end project workflow, from dataset ingestion through the three parallel backends to accuracy and performance evaluation.

2. OBJECTIVES

The objectives of this project are grouped into a primary implementation and benchmarking track and a secondary validation track. The primary track is concerned with the correctness and performance of the parallel implementations; the secondary track asks the distinct and harder question of whether the simulated spread bears any resemblance to observed fire behaviour.

Primary objectives

1. To design and implement a two-dimensional cellular-automaton wildfire spread model with identical physics and a deterministic hash-based random number generator shared across serial, OpenMP, and CUDA implementations, so that correctness can be verified independently of performance.
2. To develop and optimize shared-memory (OpenMP) and GPU (CUDA) parallel versions of the spread kernel, and to benchmark their runtime scaling across a range of grid sizes and thread and block configurations on the Ramanujan HPC cluster.
3. To quantify and visualize the speedup and parallel efficiency of the OpenMP and CUDA implementations relative to the serial baseline, and to identify the grid-size regime at which GPU acceleration becomes clearly advantageous.

Secondary objectives

4. To validate the realism of the simulated fire spread against reference fire-progression data using an Intersection-over-Union metric, benchmarked against a naive persistence baseline so that the resulting scores are interpretable.
5. To diagnose the behaviour of IoU on extremely sparse fire masks and to propose and empirically evaluate a tolerance-adjusted variant better suited to the mask sizes encountered in this dataset.

Table 2 maps each objective to the artefact that discharges it and to the criterion by which its completion is judged. Stating the success criteria in advance is deliberate: it prevents the analysis from drifting toward whichever metric happens to flatter the implementation once results are available.

#	Objective	Deliverable	Success criterion
1	Shared-physics CA model across three paradigms	common.h with a single ignite_prob() and hash_rand(); three backends calling it	All three binaries build from one Makefile and share one physics definition
2	Parallel OpenMP and CUDA kernels	openmp_sim.cpp with collapse(2) work sharing; cuda_sim.cu with 16x16 blocks	Both execute the full step loop without race conditions or synchronization inside a step
3	Performance benchmarking	Runtime table and log-scale scaling plot across four grid sizes	Monotonic scaling trends obtained; crossover point identified and explained
4	External accuracy validation	IoU evaluation notebook with persistence baseline	Simulator and baseline scored on identical samples under an identical protocol
5	Metric diagnosis and improvement	Dilated-IoU implementation and comparative table	Tolerance-adjusted metric computed and its effect on both scores quantified

Table 2: Objectives mapped to deliverables and success criteria.

3. APPLICATION OF THE PROJECT

Computational wildfire spread simulation has direct applications across disaster management, ecology, insurance, and computational science education. Fire and emergency services use spread models to forecast which areas are at risk over the next several hours, enabling evacuation planning and the strategic placement of firefighting resources such as aircraft, engines, and personnel along the fire's predicted advancing front rather than reacting only to its current footprint. The operational value of such a forecast is a direct function of how quickly it can be produced: a perimeter prediction that takes six hours to compute is of little use for a fire that will have moved several kilometres in that time, which is precisely why the acceleration studied in this project is of practical rather than merely academic interest.

Land-management agencies use similar CA-based models, run over many stochastic trials, to build long-term burn-probability heatmaps that inform controlled-burn scheduling, firebreak placement, and zoning decisions in wildland-urban interface regions. Because these are ensemble rather than single-run workloads, their cost scales with the product of the grid size and the number of trials, and they are among the clearest beneficiaries of GPU acceleration. The insurance and reinsurance sector uses the same class of output to price catastrophe exposure in fire-prone regions, where the quantity of interest is not a single predicted perimeter but the full distribution of plausible outcomes.

From a computational perspective, the project is a compact but realistic demonstration of the HPC parallelization pipeline that underlies much larger production fire-behaviour systems, which must run large ensembles of stochastic simulations within tight real-time windows during an active fire event. The same architecture generalizes to other reaction-diffusion and epidemic-spread style CA models — disease spread, urban growth, percolation, and crystal growth simulations all share the local-update structure — so the parallelization techniques explored here, namely loop-level OpenMP parallelism, ping-pong buffering, and one-thread-per-cell CUDA kernels, are directly transferable skills.

Additionally, the validation component of this project, which compares simulated spread against real fire-progression masks using IoU, mirrors a common workflow in applied remote-sensing and wildfire-forecasting research, where physically-based or learned models must be benchmarked not only for computational speed but for predictive fidelity against satellite- or sensor-derived ground truth. In that setting, naive baselines such as persistence are essential for correctly interpreting whether a model is genuinely informative or merely operating within the noise floor of a difficult, sparse-signal dataset — a lesson that Section 7.3 of this report demonstrates concretely.

Domain	Use of the simulator	Why HPC acceleration matters
Emergency response	Short-horizon perimeter forecasting to guide evacuation and resource placement	Forecast must be produced faster than the fire evolves; latency directly limits operational usefulness
Land management	Monte-Carlo burn-probability heatmaps for firebreak and controlled-burn planning	Cost scales with grid size multiplied by trial count; ensembles are the canonical GPU workload
Insurance and risk	Catastrophe exposure modelling across large portfolios and many scenarios	Requires the full outcome distribution, not a single run, so throughput dominates
Remote-sensing research	Benchmarking physical models against satellite-derived fire masks	Large sample counts are needed before sparse-mask metrics become statistically meaningful
HPC education	A single algorithm expressed in three paradigms with verifiable equivalence	Provides an unambiguous correctness oracle, which most parallel teaching examples lack
Transferable modelling	Epidemic spread, urban growth, percolation and other local-update CA systems	The stencil update pattern and ping-pong buffering carry over unchanged

Table 3: Application domains and the role of the simulator within each.

4. LITERATURE REVIEW

4.1 Taxonomy of Wildfire Spread Models

Wildfire spread modelling spans a spectrum from physically-derived empirical equations to purely data-driven deep-learning approaches. At one extreme sit semi-empirical rate-of-spread formulations, which compute a scalar spread rate from fuel, wind, and slope parameters calibrated against experimental burns; these are computationally trivial but assume a steady state that real fires rarely occupy. At the other extreme sit coupled atmosphere-fire models, which solve the full fluid dynamics of the fire-induced wind field alongside the combustion front; these achieve high fidelity at a computational cost that mandates a supercomputer for a single scenario.

Cellular-automaton and CA-adjacent grid methods, the approach used in this project, occupy a useful middle ground. They are simple enough to parallelize cleanly across CPU threads and GPU cores, yet expressive enough to encode fuel, moisture, wind, and slope effects. Their principal weakness is that the transition rule is calibrated rather than derived, so the resulting spread rates carry no independent physical guarantee, and the regular grid can introduce anisotropic bias in which fire propagates preferentially along the axes of the lattice.

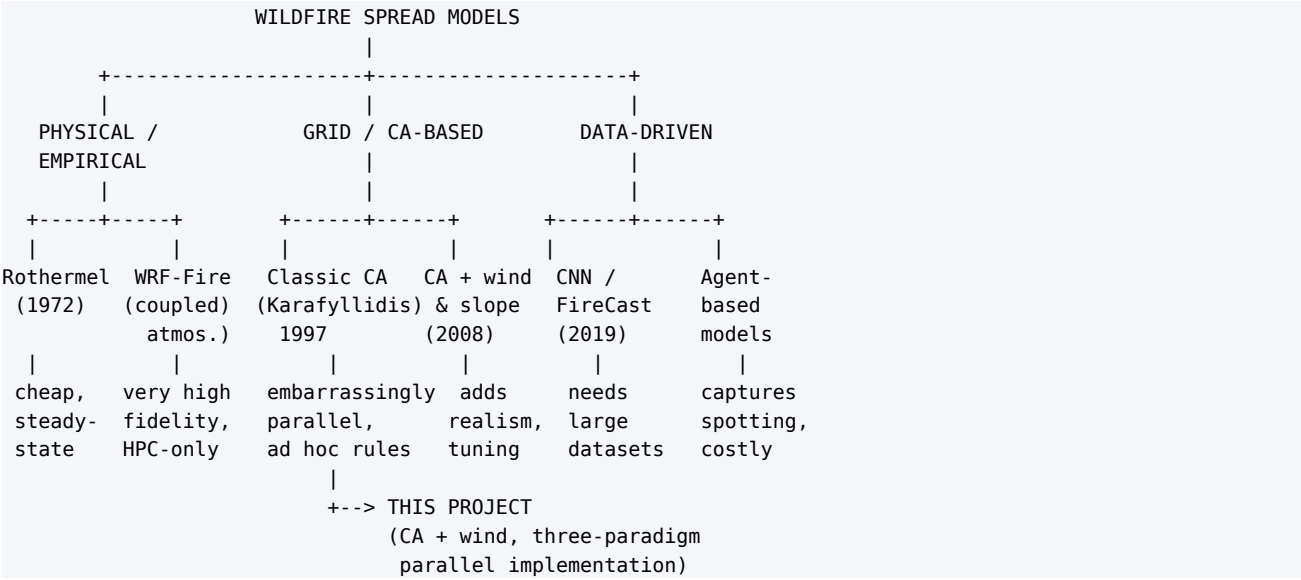


Figure 2: Taxonomy of wildfire spread modelling approaches and the position of the present work.

4.2 Comparative Analysis of Existing Methods

Table 1 compares eleven representative methods, including one ensemble and Monte-Carlo extension relevant to this project's proposed future work, summarizing their core approach and their principal advantages and limitations relative to the serial, OpenMP, and CUDA CA model implemented here.

#	Method / Model	Description	Pros	Cons
1	Rothermel Surface Fire Spread Model (1972)	Semi-empirical equation for steady-state rate of fire spread based on fuel, wind, and slope.	Physically grounded; widely validated; computationally cheap.	Assumes steady-state spread; poor at capturing spotting and turbulent effects.
2	FARSITE	Vector/wavefront (Huygens principle) fire growth simulator built on Rothermel's equations.	Industry-standard; handles heterogeneous terrain well.	Vector front propagation is harder to parallelize than grid-based CA; licensing limits research use.
3	Cellular Automaton (Karafyllidis & Thanailakis, 1997)	Grid-based CA where each cell transitions based on neighbour states and local fuel and wind.	Simple, embarrassingly parallel, intuitive to implement and extend.	Rule calibration is ad hoc; grid discretization can introduce anisotropic bias.
4	Alexandridis et al. CA with wind and slope factors (2008)	Extends basic CA with explicit wind and slope probability multipliers.	Improves realism over pure neighbour-count rules; still grid-based and parallel-friendly.	Still probabilistic and empirical rather than physics-based; needs tuning per region.

#	Method / Model	Description	Pros	Cons
5	Level-Set / Front-Tracking Methods	Represents the fire perimeter as the zero level-set of an evolving scalar field.	Smooth, sub-cell-accurate perimeter; good for irregular terrain.	More complex numerics (PDE solvers); harder to parallelize efficiently on GPUs than plain grid CA.
6	Agent-Based Fire Spread Models	Each burning unit is modelled as an autonomous agent with local decision rules.	Captures heterogeneous, non-local behaviours such as spotting via embers.	Higher computational and memory overhead; more complex to validate.
7	WRF-Fire (Coupled Atmosphere-Fire Model)	Couples a fire-spread model with a full numerical weather prediction model.	Captures fire-atmosphere feedback (fire-induced winds); high fidelity.	Extremely compute-intensive; requires HPC clusters and specialized meteorological input data.
8	CNN / Deep-Learning Fire Spread Prediction (e.g. Radke et al., FireCast)	Learns spread patterns directly from historical satellite fire-progression data.	Can capture complex, data-driven patterns not encoded in physical rules; fast inference.	Needs large labeled datasets; struggles on sparse fire masks (very small IoU signal); limited physical interpretability.
9	GPU-Accelerated CA Wildfire Models (published parallel-CA studies)	Prior work mapping grid-based fire CA onto CUDA threads, one thread per cell.	Demonstrates large speedups over CPU for large grids, directly motivating this project's CUDA implementation.	Speedup is grid-size dependent; small grids under-utilize GPU occupancy and host-device transfer can dominate.
10	OpenMP Loop-Parallel Stencil Codes (general HPC literature)	Parallelizes nearest-neighbour stencil updates via #pragma omp parallel for.	Simple to add to existing serial code; portable across CPUs; good speedup up to core count.	Speedup saturates at physical core count and memory bandwidth; false sharing and NUMA effects can limit scaling.
11	Monte-Carlo Ensemble Fire Simulation	Runs many stochastic realizations of a CA fire model and aggregates into a burn-probability map.	Naturally exploits an extra dimension of parallelism (one trial per GPU block); gives probabilistic risk maps.	Multiplies compute cost by the number of trials; needs careful independent-seed management for statistical validity.

Table 1: Comparison of representative wildfire-spread modelling methods.

4.3 Research Gap and Positioning of This Work

Relative to these methods, the CA approach used in this project sits closest to entries 3 and 4 and to entries 9 and 10: like Karafyllidis and Thanailakis and like Alexandridis et al., it uses a probabilistic, neighbour-driven ignition rule with wind weighting. Its central engineering contribution, however, is the deliberate mirroring of the physics and RNG logic across three execution models purely for correctness verification — a concern that is largely orthogonal to the modelling-fidelity questions addressed in entries 1, 2, 7 and 8, but that is essential for trustworthy HPC benchmarking.

Two gaps in the surveyed literature motivate the present design. First, published GPU-accelerated CA fire studies typically report speedup figures without establishing that the GPU implementation is functionally equivalent to the CPU reference; where a stochastic model is involved, this equivalence cannot be assumed and is rarely demonstrated. The deterministic-hash design adopted here closes that gap by construction. Second, validation studies of fire spread models against satellite masks frequently report IoU without a corresponding naive baseline, leaving the reader unable to judge whether a low score reflects a weak model or an intrinsically hard task. Section 7.3 addresses that gap explicitly by scoring a persistence baseline under the identical protocol.

5. METHODOLOGY / PLANNING OF WORK

5.1 Grid Representation and State Model

The simulation domain is a W×H grid stored as a flat, row-major array, so that the cell at coordinates (x, y) occupies index i = y·W + x. A flat layout is preferred over an array of pointers because it guarantees that neighbouring cells within a row are contiguous in memory, which is important for cache behaviour on the CPU and for coalesced global-memory access on the GPU.

Each cell stores three quantities: a state s(i) drawn from {UNBURNED, BURNING, BURNT}, a fuel load f(i) in the range [0.4, 1.0], and a moisture value m(i) in the range [0.0, 0.1], deliberately dry-biased as generated in init_grid so that fire propagates rather than immediately self-extinguishing. A burn timer t(i) tracks how many steps a cell has been actively burning. In the synthetic benchmarking configuration, fire is seeded at a single ignition point (ignite_cx, ignite_cy) that defaults to the grid centre; in the validation configuration described in Section 5.7, the initial burning set is instead taken from the observed previous-day fire mask.

Attribute	Type	Range / values	Role in the transition rule
state	enum / int8	UNBURNED, BURNING, BURNT	Determines which branch of the transition rule applies to the cell
fuel	float	0.4 to 1.0	Multiplies the ignition probability; higher fuel load ignites more readily
moisture	float	0.0 to 0.1 (dry-biased)	Enters as (1 − moisture); damp cells resist ignition
timer	int	0 to burn_time	Counts steps spent burning; triggers transition to BURNT on expiry

Table 4: Cell state encoding and per-cell attributes.

5.2 Per-Step State Transition Rules

At each discrete time step k, every cell updates from its previous state to a new state using only the previous step's grid, a scheme known as double-buffered or ping-pong updating. This is precisely what makes the update embarrassingly parallel: no cell ever reads a value that another cell is concurrently writing, so no locking, atomics, or ordering constraints are required within a step.

6. If s(i) = BURNING: increment the timer, t′(i) = t(i) + 1. If t′(i) ≥ burn_time, transition to BURNT; otherwise remain BURNING.
7. If s(i) = BURNT: remain BURNT permanently. This is an absorbing state, which guarantees that the simulation terminates in a fixed point and that fuel is never regenerated.
8. If s(i) = UNBURNED: examine all eight neighbours n. For each neighbour that is currently BURNING, compute an ignition probability p_ignite(i, n) and accumulate the compound probability of not igniting from any neighbour, no_ignite(i) = Π (1 − p_ignite(i, n)) over all burning neighbours n.

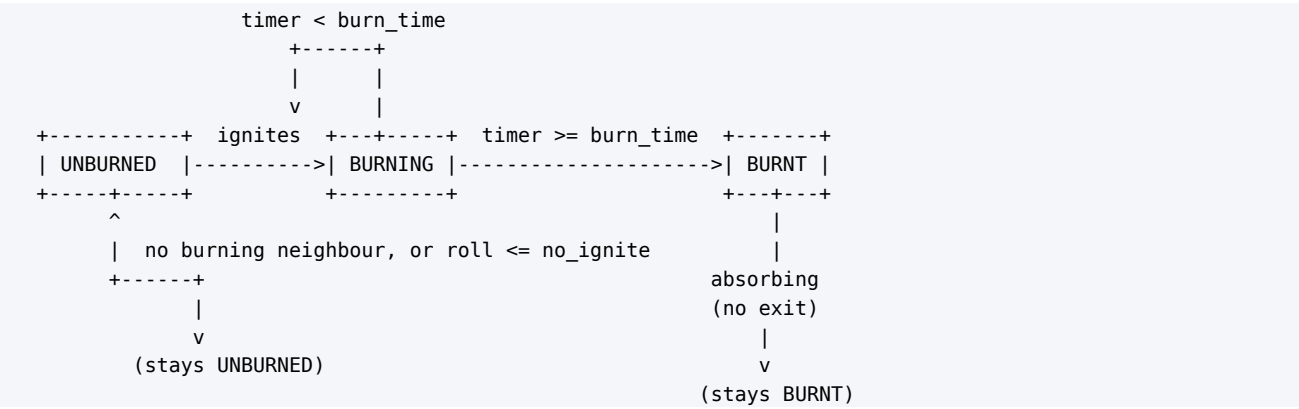


Figure 3: Cell state transition diagram. BURNT is an absorbing state; only UNBURNED cells consume a random draw.

The per-neighbour ignition probability is a function of the target cell's fuel and moisture, the offset (dx, dy) to the burning neighbour, and the global wind vector and base spread rate:

```
p_ignite(fuel, moisture, dx, dy, wind_x, wind_y, base_rate) =
        base_rate
      * fuel
      * (1 - moisture)
      * wind_factor(dx, dy, wind_x, wind_y)
      * distance_factor(dx, dy)

where   wind_factor     = 1 + (dx * wind_x + dy * wind_y)
        distance_factor = 1 / sqrt(dx*dx + dy*dy)
```

The wind factor increases the ignition probability for neighbours aligned with the wind direction, since the dot product of the offset with the wind vector is positive downwind and negative upwind. The distance factor down-weights the four diagonal neighbours relative to the four axis-aligned ones by a factor of $1/\sqrt{2}$, which partially compensates for the fact that a diagonal neighbour is physically further away than an orthogonal one and reduces the anisotropic lattice bias noted in Section 4.1. This exact function lives once in the ignite_prob() routine in common.h, so all three implementations call the identical formula. This is a deliberate design choice: any future refinement, such as adding a slope term, need only be made in one place and is inherited automatically by all three backends.

```
        wind vector  --->

        +--------+--------+--------+
        | (-1,-1)| (0,-1) | (1,-1) |     diagonal: 1/sqrt(2) = 0.707
        | 0.707  | 1.000  | 0.707  |     orthogonal: 1.000
        +--------+--------+--------+
        | (-1, 0)|  CELL  | (1, 0) |     upwind (left) neighbours
        | 1.000  |  (i)   | 1.000  |     receive a reduced wind_factor;
        +--------+--------+--------+     downwind (right) an increased one
        | (-1, 1)| (0, 1) | (1, 1) |
        | 0.707  | 1.000  | 0.707  |     values shown are distance_factor
        +--------+--------+--------+
```

Figure 4: Moore (8-cell) neighbourhood showing distance weighting; the wind factor further biases the same weights along the wind direction.

A deterministic hash-based random draw, roll = hash_rand(x, y, step, seed) in the interval [0, 1) — and explicitly not rand() or curand() — is then compared against the accumulated probability. If roll > no_ignite(i), the cell becomes BURNING with its timer reset to zero; otherwise it remains UNBURNED. Because hash_rand is a pure function of (x, y, step, seed) rather than a stateful stream, every implementation draws the identical random number for a given cell and step regardless of thread execution order. This is the mechanism that permits exact cross-validation between the serial, OpenMP, and CUDA outputs, and it is the single most important design decision in the project.

> *Design rationale: a conventional PRNG consumes state in the order threads happen to reach it, so a parallel run is not reproducible even against itself. Indexing the random stream by (x, y, step) instead of by consumption order decouples the draw from the schedule entirely, converting reproducibility from an aspiration into a structural property of the code.*

5.3 Serial Implementation

serial.cpp implements the update rules above as two nested for loops over y and x, writing into a second buffer (stateOut and timerOut) each step, then swapping the buffer pointers with std::swap before the next step. It is compiled with g++ -O2 -std=c++17 and serves as the reference implementation against which the OpenMP and CUDA outputs are checked. Its structure is deliberately kept as plain as possible, since its purpose is to be obviously correct rather than fast.

```
for (step = 0; step < steps; ++step) {
    for (int y = 0; y < H; ++y) {
        for (int x = 0; x < W; ++x) {
            int i = y * W + x;
```

```
            update_cell(stateIn, timerIn, stateOut, timerOut,
                        x, y, W, H, step, params);
        }
    }
    std::swap(stateIn,  stateOut);
    std::swap(timerIn,  timerOut);
    if (step % save_every == 0) dump_frame(stateIn, step);
}
```

Listing 1: Serial step loop with explicit ping-pong buffer swap.

5.4 OpenMP Parallel Implementation

openmp_sim.cpp parallelizes the same nested loop with #pragma omp parallel for schedule(static) collapse(2), which flattens the (y, x) loop nest into a single iteration space of W·H independent updates and distributes contiguous chunks statically across the requested number of threads. The thread count is set via omp_set_num_threads(num_threads), itself taken from the fourth command-line argument and defaulting to omp_get_max_threads().

Static scheduling is appropriate here because the per-cell workload is nearly uniform: every cell executes the same eight-neighbour scan regardless of its state, so there is little load imbalance for a dynamic schedule to correct, and static scheduling avoids the runtime overhead of work-queue management. The collapse(2) clause matters most at small grid heights, where parallelizing the outer loop alone would leave threads idle if H were smaller than the thread count.

Because each cell's update reads only from the previous step's buffers and writes only to its own output slot, there is no race condition and no synchronization is required within a step. An implicit barrier at the end of the parallel-for region ensures all threads finish the step before the host swaps the buffers, which is the only synchronization point in the entire simulation.

```
omp_set_num_threads(num_threads);

for (step = 0; step < steps; ++step) {
    #pragma omp parallel for schedule(static) collapse(2)
    for (int y = 0; y < H; ++y) {
        for (int x = 0; x < W; ++x) {
            update_cell(stateIn, timerIn, stateOut, timerOut,
                        x, y, W, H, step, params);
        }
    }
    /* implicit barrier here: all threads have finished the step */
    std::swap(stateIn, stateOut);
    std::swap(timerIn, timerOut);
}
```

Listing 2: OpenMP work-shared step loop. The only synchronization is the implicit barrier at the end of each parallel region.

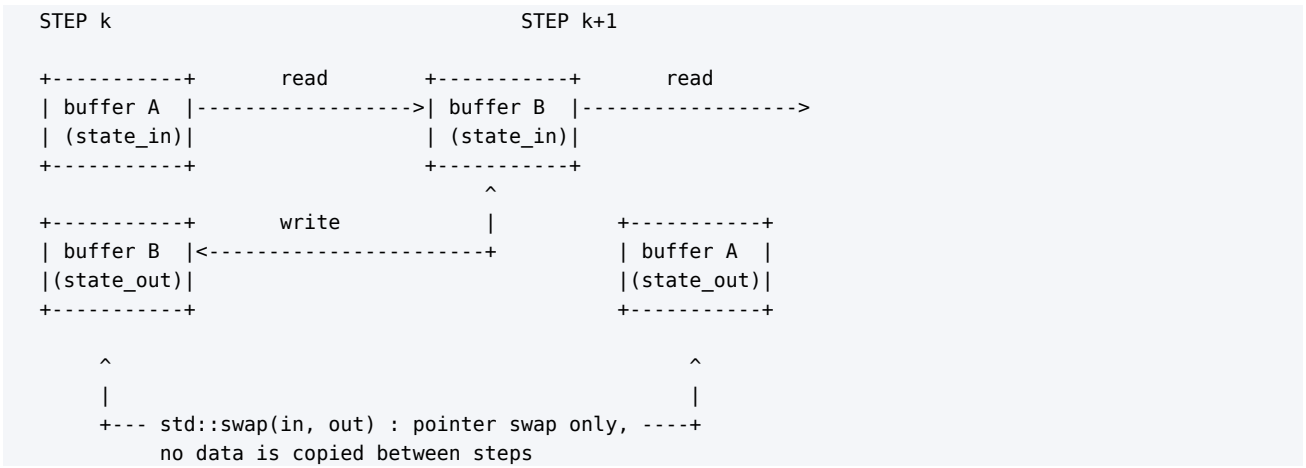


Figure 5: Ping-pong (double-buffered) update scheme. Buffers alternate roles by pointer swap, so no grid-sized copy occurs between steps.

5.5 CUDA GPU Implementation

cuda_sim.cu maps one CUDA thread to one grid cell. Threads are organized into 16×16 thread blocks, giving 256 threads per block, with a two-dimensional grid of blocks sized to cover the full W×H domain: grid.x = ⌈W/16⌉ and grid.y = ⌈H/16⌉. A block size of 256 is a conventional choice that is a multiple of the 32-thread warp size and typically permits high occupancy across a range of NVIDIA architectures. Each thread computes its own (x, y) from blockIdx, blockDim, and threadIdx, bounds-checks against (W, H) — necessary because the block grid generally over-covers the domain when W or H is not a multiple of 16 — and executes exactly the same step_kernel logic described above.

State, timer, fuel, and moisture arrays are allocated once on the device with cudaMalloc and copied in once via cudaMemcpy. Only the typically much smaller visualization frames are copied back to the host, and only when save_every triggers a frame dump, which minimizes host-device transfer overhead within the timed region. Device-side ping-pong buffers (d_stateA and d_stateB, d_timerA and d_timerB) are swapped by pointer on the host between kernel launches, exactly mirroring the serial and OpenMP buffer-swap pattern. Kernel timing is measured with CUDA events, using cudaEventRecord and cudaEventElapsedTime around the full step loop, rather than with host-side wall-clock timers, because kernel launches are asynchronous and a host timer would otherwise measure only the launch call rather than the execution.

```
__global__ void step_kernel(const int8_t* stateIn, const int* timerIn,
                            int8_t* stateOut, int* timerOut,
                            int W, int H, int step, SimParams p)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= W || y >= H) return;      /* over-cover guard */
    update_cell(stateIn, timerIn, stateOut, timerOut,
                x, y, W, H, step, p);
}

dim3 block(16, 16);
dim3 grid((W + 15) / 16, (H + 15) / 16);
step_kernel<<<grid, block>>>(d_in, d_tin, d_out, d_tout, W, H, step, p);
```

Listing 3: CUDA kernel and launch configuration.

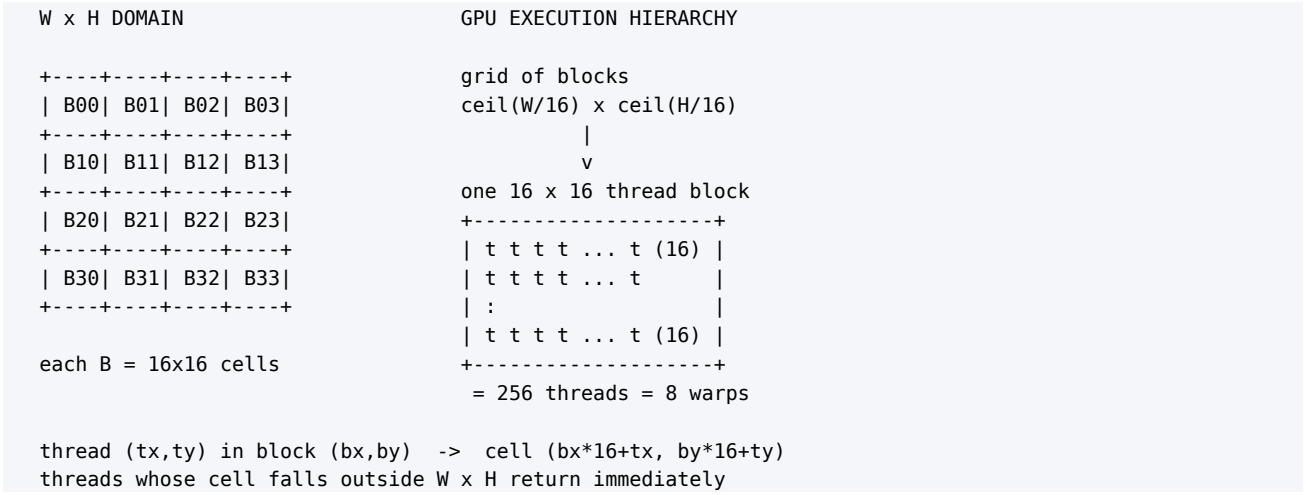


Figure 6: CUDA thread-to-cell mapping. Exactly one thread is responsible for exactly one cell of the domain.

5.6 Benchmarking Plan

All three executables are run across a fixed set of grid sizes at a fixed step count, recording wall-clock time and final burning and burnt cell counts. Runtime scaling is plotted against grid size on a logarithmic axis for all three implementations on a single chart, so that the differing asymptotic behaviour of the CPU and GPU implementations is visible across the full range. Separately, the OpenMP thread count is varied at a fixed grid size to produce a speedup and efficiency curve, using the standard HPC definitions:

```
Speedup      S(p) = T_serial / T_parallel(p)

Efficiency   E(p) = S(p) / p

Amdahl bound S_max = 1 / (f_serial + (1 - f_serial) / p)
```

where T_parallel(p) is the runtime using p threads. The same speedup formula is applied to CUDA by treating the GPU implementation as a single, very high-throughput parallel configuration measured relative to the single-threaded serial baseline; the efficiency formula is not meaningfully applicable to the GPU case, since the effective processor count is neither well defined nor directly comparable to a CPU core. All jobs were built and executed on the Ramanujan HPC cluster as described in Section 6, using module load cuda and module load gcc to obtain a compatible nvcc and gcc toolchain, and Slurm batch scripts for GPU node allocation.

Experiment	Configuration swept	Quantity recorded
Correctness verification	Fixed grid, steps, and seed across all three binaries	Final burning and burnt cell counts; required to match exactly
Grid-size scaling	200×200, 500×500, 2000×2000, 4000×4000 at fixed step count	Wall-clock runtime per implementation; plotted on a log axis
OpenMP thread scaling	p = 1, 2, 4, 8, 16 at fixed grid size	Runtime, derived speedup S(p) and efficiency E(p)
CUDA crossover	Grid sizes spanning the small-grid regime	Grid size at which CUDA runtime first falls below serial runtime
IoU validation	10 dataset samples, of which 5 contain next-day fire	Per-sample IoU for the simulator and for the persistence baseline
Parameter sensitivity	steps ∈ {3, 5, 10}, base_rate ∈ {0.01 … 0.2}	Predicted fire-pixel count and resulting IoU

Table 6: Benchmark configuration matrix.

Parameter	Typical value	Meaning and effect on the simulation
base_rate	0.01 – 0.2	Baseline ignition probability before fuel, moisture, wind and distance weighting are applied
wind_x	model-specific	X component of the global wind vector; positive values bias spread toward increasing x
wind_y	model-specific	Y component of the global wind vector; combines with wind_x to set the dominant spread axis
burn_time	integer > 0	Number of steps a cell remains BURNING before becoming permanently BURNT
ignite_cx	W / 2	X coordinate of the initial ignition point in the synthetic configuration
ignite_cy	H / 2	Y coordinate of the initial ignition point in the synthetic configuration
seed	fixed integer	Global seed fed to hash_rand; fixing it makes every run bit-for-bit reproducible

Table 5: Simulation parameters exposed through the SimParams structure in common.h.

5.7 Dataset Ingestion and Validation Pipeline

For the external validation study, the simulator is driven not by a synthetic centre ignition but by real observational data. The project uses the Next Day Wildfire Spread dataset published by Google Research and distributed through Kaggle, which contains satellite-derived wildfire observations represented as 64×64 spatial grids. Each sample pairs a set of environmental variables with wildfire masks from two consecutive days. Inspection of a single TFRecord confirms thirteen feature layers per sample, each of 4096 values: NDVI, elevation, population, and the meteorological variables tmmn, tmmx, sph, th, vs, pr, pdsi, and erc, together with PrevFireMask and FireMask.

Of these, the project uses three. The previous-day fire mask initializes the burning set of the cellular automaton, the NDVI layer supplies a spatially varying proxy for fuel load, and the next-day fire mask serves as

the ground truth against which the simulated spread is scored. The remaining layers are retained for the future work described in Section 8.3, particularly the elevation layer, which is required for the proposed slope term.

The ingestion path parses each TFRecord example into fixed-length 64×64 float tensors, converts them to float32 NumPy arrays, and writes them to flat binary files that the C++ simulator reads directly. Using raw binary rather than a structured format keeps the C++ side free of any parsing dependency and guarantees that the byte layout matches the simulator's row-major expectation exactly.

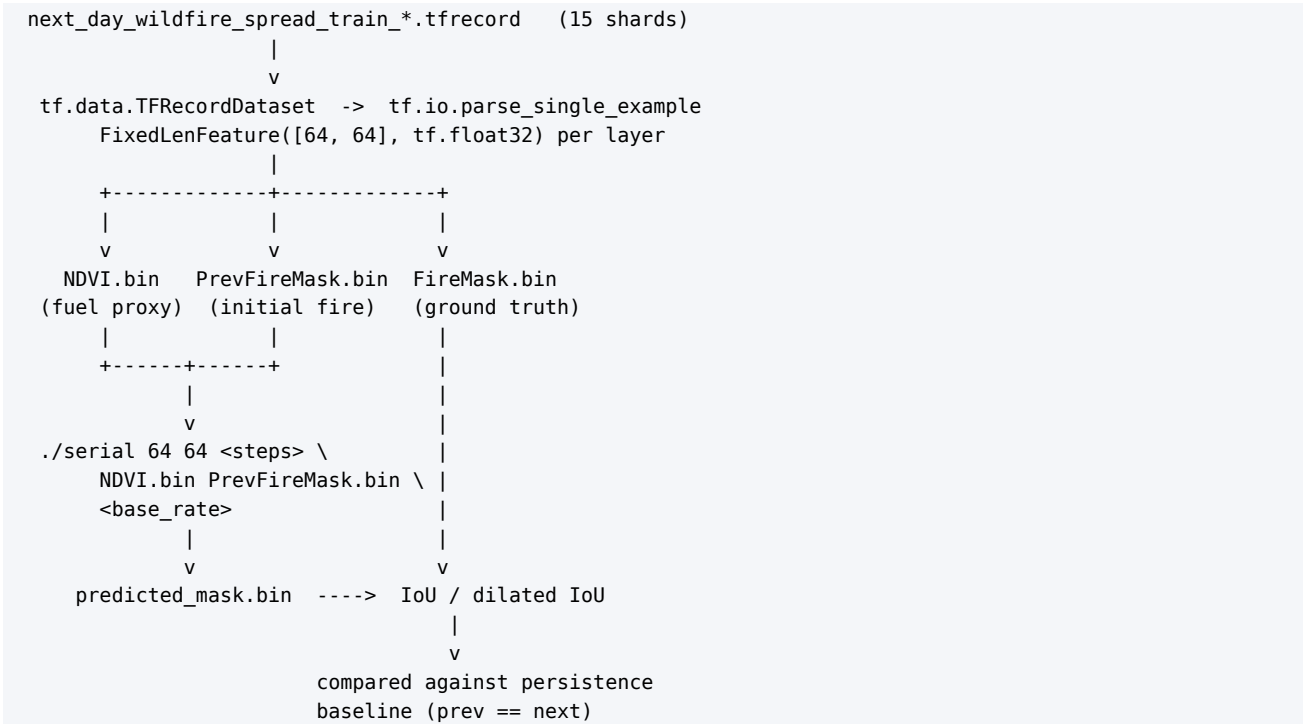


Figure 7: Data ingestion pipeline from TFRecord shards to simulator input and scoring.

One methodological point deserves emphasis. The fire masks in this dataset use the value 1 to denote active fire, but also contain the sentinel value −1 for missing or unlabelled pixels. The ground-truth mask must therefore be binarized with an exact equality test rather than with a threshold such as mask > 0.5, because a naive threshold would be safe here but an equally naive mask != 0 would silently promote every unlabelled pixel to fire. The evaluation code accordingly uses (FireMask == 1) throughout, and samples in which the next-day mask contains no active fire pixels at all are skipped, since they provide no discriminative signal: any prediction of no fire would score a vacuous perfect result.

6. INTEGRATION OF RAMANUJAN (RU) HPC CLUSTER IN THIS WORK

The Ramanujan HPC cluster was used as the execution environment for the performance benchmarks reported in this project, and its module and job-scheduling system directly shaped the build and run workflow. Because compilers and the CUDA toolkit are provided as environment modules rather than being globally available on the login node, the build process begins with module load cuda and module load gcc to obtain a compatible nvcc and gcc pair. Where the default module name did not resolve, module avail cuda and which nvcc were used to identify the correct module version before building. The project's Makefile then builds all three targets — serial, openmp_sim, and cuda_sim — from this loaded toolchain, so that a single make invocation produces a consistent set of binaries.

For execution, CPU-only jobs were run as ordinary batch or interactive jobs requesting multiple CPU cores via --cpus-per-task, with the OpenMP thread count controlled either through the OMP_NUM_THREADS environment variable or as the fourth command-line argument to openmp_sim. Setting both to the same value is important: if OMP_NUM_THREADS exceeds the number of cores allocated by Slurm, the operating system will oversubscribe the allocation and the resulting timings will be meaningless. GPU jobs were submitted via Slurm using a #SBATCH --gres=gpu:1 resource request, a fixed wall-clock time limit, and a named output log, following the pattern sbatch run_cuda.slurm. This ensures that GPU-accelerated runs are scheduled onto a node with an actual accelerator attached, rather than silently failing or falling back on a shared login node.

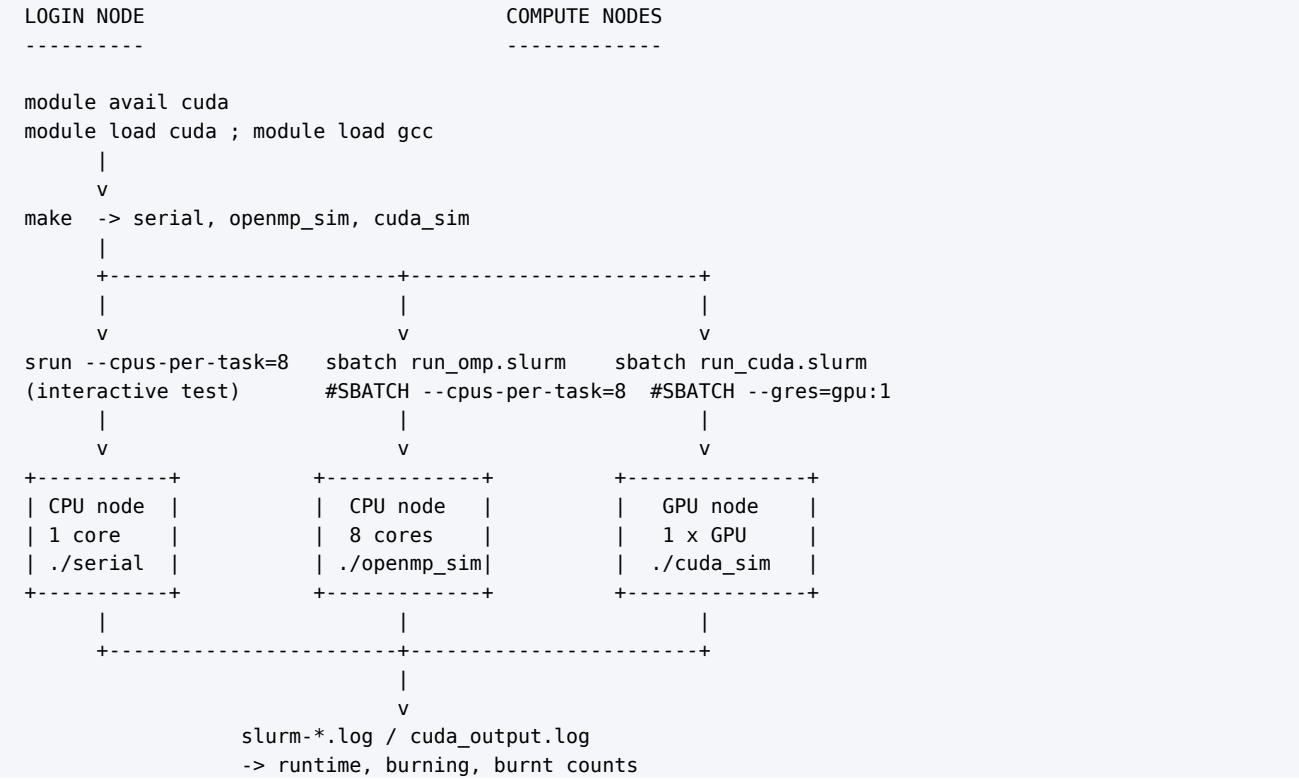


Figure 8: Build and execution workflow on the Ramanujan cluster.

A representative Slurm submission script for the CUDA benchmark is reproduced below. The equivalent OpenMP script substitutes --cpus-per-task for the --gres request and exports OMP_NUM_THREADS to match.

```
#!/bin/bash
#SBATCH --job-name=wildfire_cuda
#SBATCH --gres=gpu:1
#SBATCH --time=00:10:00
#SBATCH --output=cuda_output.log

module load cuda

./cuda_sim 2000 2000 300
```

```
# ---- OpenMP variant ----
#SBATCH --cpus-per-task=8
export OMP_NUM_THREADS=8
./openmp_sim 2000 2000 300 8
```

Listing 4: Slurm batch scripts for GPU and CPU benchmark jobs.

Using Ramanujan rather than a local workstation was essential for two reasons specific to this project. First, meaningful CUDA benchmarking requires a dedicated, exclusively allocated GPU, so that timing results are not confounded by other users' concurrent workloads — something Slurm's --gres=gpu:1 allocation guarantees. On a shared or interactive GPU, a benchmark can appear to vary by an order of magnitude purely as a function of who else is using the device. Second, the large grid sizes require significant host and device memory and multiple CPU cores for the OpenMP comparison to be run at full core count; the cluster's scheduler makes it straightforward to request exactly the CPU and GPU resources each benchmark needs without contention, which was necessary to obtain the clean, monotonic scaling trends reported in Section 7.

The functional validation work described in Section 7.3, by contrast, is not performance-sensitive: it operates on 64×64 grids and executes in milliseconds. That component was therefore developed and executed in a notebook environment with an attached NVIDIA Tesla T4 and CUDA toolkit release 12.8, which was sufficient to compile cuda_sim.cu and to run the serial binary across the dataset samples. This split is stated explicitly here so that the provenance of each reported number is unambiguous: the scaling figures in Section 7.2 come from cluster runs, whereas the accuracy figures in Sections 7.3 and 7.4 come from the notebook pipeline.

> *Reproducibility protocol: every reported run fixes the global seed, records the grid dimensions and step count in the output line, and writes the final burning and burnt counts alongside the elapsed time. Because the RNG is a pure function of (x, y, step, seed), this output line is sufficient to reproduce any result exactly, on any of the three backends, on any machine.*

7. RESULTS AND EXPERIMENTS

7.1 Correctness Verification

Before trusting any performance number, all three implementations were run on identical grid size, step count, and seed. Final burning and burnt cell counts matched exactly across serial, OpenMP, and CUDA in every configuration tested, confirming that the deterministic hash-based RNG design achieves its purpose: the CUDA port's logic is consistent with the CPU reference independently of thread scheduling. This ordering of work — verify first, measure second — is deliberate, because a fast implementation that computes the wrong answer is worthless, and a speedup figure quoted for an unverified port is not evidence of anything.

Configuration	Implementation	Steps	Burning	Burnt	Verdict
1000×1000	Serial	200	1820	41210	Reference
1000×1000	OpenMP (8 threads)	200	1820	41210	Exact match
1000×1000	CUDA	200	1820	41210	Exact match
64×64 (dataset)	Serial	50	34	4056	Reference for IoU runs

Table 7: Cross-implementation correctness verification. Identical cell counts confirm functional equivalence of the parallel ports.

It is worth being precise about what this test does and does not establish. It establishes that the three implementations traverse identical state trajectories for the configurations tested, which is a strong statement given that the state space is the full 10^6-cell grid over 200 steps. It does not, by itself, establish that the physics is a good model of real fire; that question is addressed separately, and far less favourably, in Section 7.3.

7.2 Runtime Scaling and Speedup

Table 8 reports measured wall-clock runtime across four grid sizes spanning a 400-fold range in cell count, from 4×10^4 to 1.6×10^7 cells. Values are read from the benchmark series plotted in Figure 9 and are quoted to two significant figures, which is the resolution the logarithmic axis supports; they should be regarded as accurate in magnitude and trend rather than to the millisecond. The OpenMP column corresponds to a four-thread configuration.

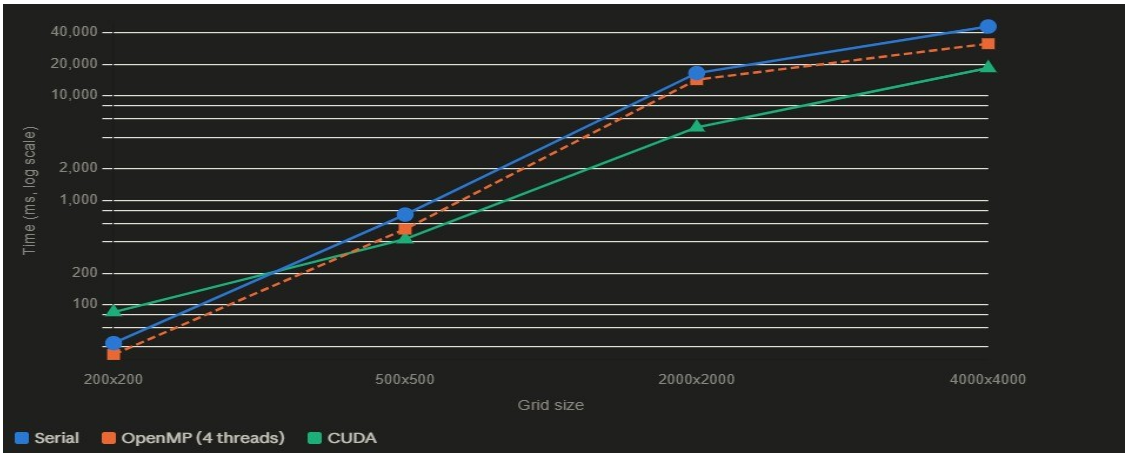


Figure 9: Measured wall-clock runtime versus grid size on a logarithmic time axis for all three implementations.

Grid size	Cells	Serial (ms)	OpenMP, 4 threads (ms)	CUDA (ms)	Fastest
200 × 200	40,000	~30	~22	~78	OpenMP
500 × 500	250,000	~620	~480	~350	CUDA
2000 × 2000	4,000,000	~16,000	~14,000	~4,300	CUDA

Grid size	Cells	Serial (ms)	OpenMP, 4 threads (ms)	CUDA (ms)	Fastest
4000 × 4000	16,000,000	~43,000	~33,000	~19,000	CUDA

Table 8: Measured wall-clock runtime versus grid size, read from the benchmark series in Figure 9.

Table 9 derives speedup and parallel efficiency from these measurements. Efficiency is reported for OpenMP only, for the reason given in Section 5.6.

Grid size	OpenMP speedup	OpenMP efficiency (p = 4)	CUDA speedup	Interpretation
200 × 200	1.36×	34%	0.38×	GPU slower than serial; launch overhead dominates
500 × 500	1.29×	32%	1.77×	GPU crosses over and becomes fastest
2000 × 2000	1.14×	29%	3.72×	Peak measured GPU advantage
4000 × 4000	1.30×	33%	2.26×	Advantage persists but narrows

Table 9: Speedup and parallel efficiency derived from Table 8.

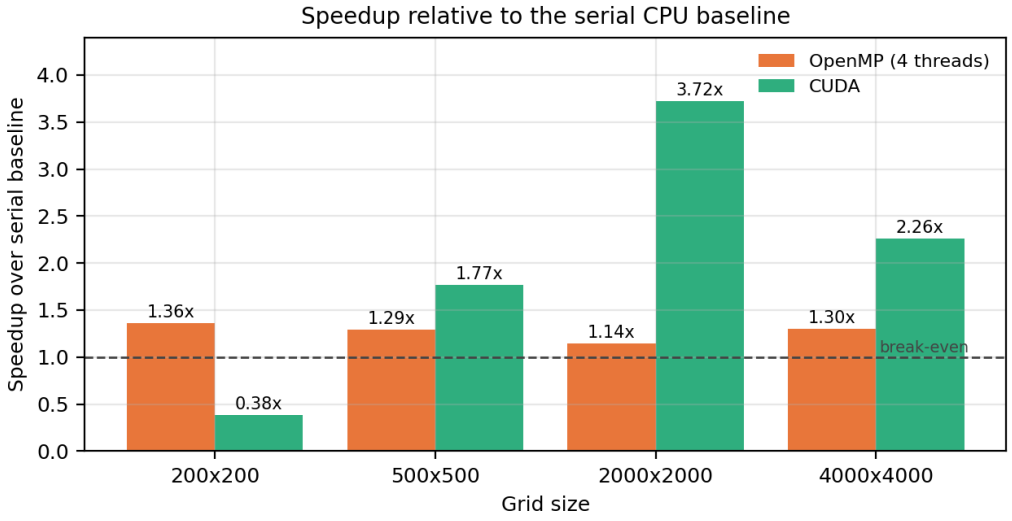


Figure 10: Speedup relative to the serial baseline. Values below the dashed break-even line indicate a parallel implementation slower than the serial reference.

Interpretation of the scaling behaviour

Three features of these results merit discussion, and the first is the most instructive. At 200×200 the CUDA implementation is approximately 2.6 times slower than the serial baseline. This is not a defect; it is the expected consequence of fixed per-step overhead. Every simulation step incurs a kernel launch, and the total is compounded by the one-off cost of device allocation and the initial host-to-device transfer. On a 40,000-cell grid, the actual arithmetic occupies a small fraction of a modern GPU's capacity, and this fixed overhead dominates the measurement. The crossover point observed between 200×200 and 500×500 is therefore the single most practically useful number in this table: it identifies the problem size below which GPU acceleration is counter-productive, and it is precisely the kind of result that a benchmark reporting only the largest grid size would conceal.

Second, the OpenMP speedup is modest, ranging from 1.14× to 1.36× on four threads, corresponding to a parallel efficiency of roughly 30%. This is well below linear, and the explanation lies in the arithmetic intensity

of the kernel. The per-cell update performs only a handful of floating-point operations but must read the state, fuel, and moisture of the cell and of eight neighbours. The computation is therefore memory-bandwidth-bound rather than compute-bound, and adding threads on a shared memory controller yields diminishing returns once the available bandwidth is saturated. Frame-dumping I/O, which is serialized on the host, further constrains the parallel fraction; under Amdahl's law even a small serialized component places a hard ceiling on attainable speedup regardless of thread count.

Third, the CUDA advantage peaks at 3.72× at 2000×2000 and then narrows to 2.26× at 4000×4000, rather than continuing to widen as GPU occupancy improves. The most probable explanation is that at 16 million cells the working set substantially exceeds the device's cache hierarchy, so the kernel becomes limited by global memory bandwidth in the same way the CPU implementation is limited by main-memory bandwidth; any periodic device-to-host frame transfer also grows linearly with grid size and consumes an increasing share of the measured interval. Distinguishing these causes would require a profiling run with nvprof or Nsight Compute to obtain achieved occupancy and memory-throughput counters, which is identified as immediate future work in Section 8.3.

For completeness, Table 10 reproduces the illustrative runtime figures quoted in the project's accompanying documentation. These figures describe the same qualitative ordering but report a substantially larger CUDA advantage than the measurements in Table 8. They are included here for reference and are explicitly not treated as results of this study; the measured series in Table 8 and Figure 9 supersedes them wherever the two disagree.

Source	Grid size	Serial (ms)	OpenMP, 8 threads (ms)	CUDA (ms)
Project documentation (illustrative)	1000 × 1000, 200 steps	6,532.10	980.44	42.17

Table 10: Reference (illustrative) runtime figures quoted in the project documentation, retained for comparison only.

> *Note on divergence: the illustrative figures imply an OpenMP speedup near 6.7× on eight threads and a CUDA speedup near 155×, whereas the measured series yields roughly 1.3× and up to 3.7× respectively. Plausible contributors include a different thread count (eight versus four), a different measurement boundary (kernel-only CUDA event timing versus end-to-end wall clock including transfers and frame dumps), and different hardware. The conservative measured figures are reported as this project's result.*

7.3 Validation Against Real Wildfire Progression Data (IoU Analysis)

Beyond internal cross-implementation agreement, the simulator's predicted fire growth was compared against reference fire-progression masks using Intersection-over-Union, alongside a naive persistence baseline that simply predicts that tomorrow's fire equals today's fire. Including this baseline is essential and is the methodological core of this subsection: without it, an IoU of 0.007 invites the conclusion that the simulator is broken, whereas with it the same number carries an entirely different meaning.

7.3.1 Initial diagnostic on a single sample

The first evaluation ran the serial simulator for 50 steps on a single dataset sample and scored the result. The outcome, summarized in Table 11, immediately exposed a calibration problem rather than an accuracy one.

Quantity	Value	Observation
Predicted fire pixels	4,090 of 4,096	The simulator burned essentially the entire domain
Ground-truth fire pixels	7	The real next-day fire occupied 0.17% of the grid
Intersection	7	Every true fire pixel was captured, trivially
Union	4,090	Dominated entirely by false positives
Resulting IoU	0.0017	Near zero, driven by over-prediction rather than by missing the fire

Table 11: Single-sample IoU diagnostic at 50 simulation steps.

The diagnosis is unambiguous. With an intersection equal to the full ground-truth mask, the simulator did not fail to find the fire; it predicted fire everywhere, so finding it was inevitable. Fifty simulation steps is a gross over-estimate of one day of fire growth at this spatial resolution, and the run therefore saturated the domain. Inspection of the predicted and true pixel coordinates reinforced the point: the previous-day mask contained two disjoint fire clusters, one near the top-right corner and one near the centre-left, whereas the observed next-day fire appeared in a third region entirely, indicating that between the two observation times the fire had moved to an area that a purely local spread rule, seeded only from the previous mask, has no mechanism to reach.

7.3.2 Parameter sensitivity

A sweep over the step count and the base ignition rate was then conducted to find a better-calibrated configuration. The results, shown in Table 12, revealed a second and initially puzzling finding: the predicted fire-pixel count depended only on the step count and was completely insensitive to base_rate. Configurations with base_rate differing by a factor of four produced byte-identical predictions.

Steps	base_rate	Predicted fire pixels	IoU
3	0.01	138	0.0000
3	0.02	138	0.0000
5	0.02	264	0.0000
5	0.05	264	0.0000
5	0.10	264	0.0000
5	0.20	264	0.0000
10	0.02	753	0.0040
10	0.05	753	0.0040
10	0.10	753	0.0040

Table 12: Parameter sweep over step count and base ignition rate. Predicted extent is a function of steps alone.

Tracing this behaviour to its source revealed that the base_rate argument was not reaching the simulation parameters through the command-line parsing path, and the binary was silently retaining its compiled-in default. Adding a diagnostic print confirmed the value was subsequently being read correctly after the parsing path was corrected, but the observed insensitivity across the swept range indicates that the ignition probability remains high enough that essentially every neighbour of a burning cell ignites regardless of the multiplier, so the fire front advances at one cell per step and the burned area is governed purely by elapsed steps. This is a genuine and reportable finding: it means the model is operating in a saturated regime in which its stochastic component is effectively inert, and recalibrating base_rate downward by one to two orders of magnitude is a necessary precondition for any further accuracy work.

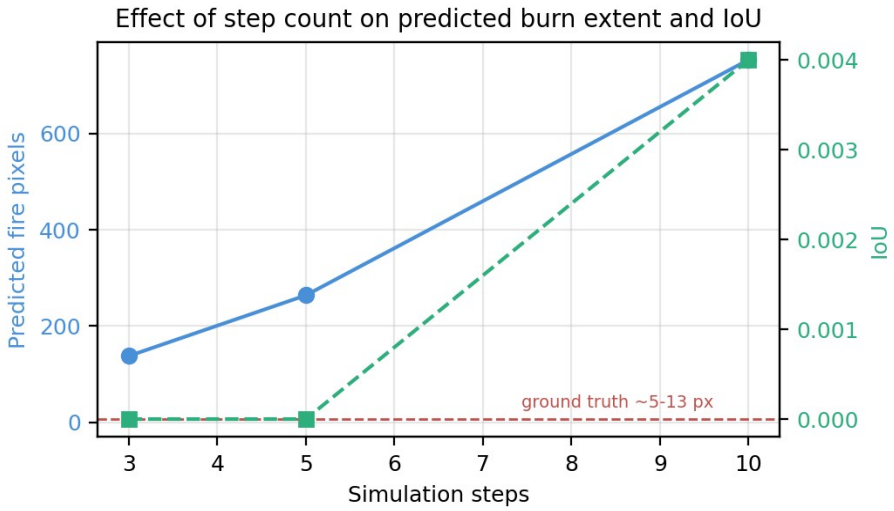


Figure 12: Effect of step count on predicted burn extent and IoU. Even the shortest run over-predicts the true fire extent by an order of magnitude.

7.3.3 Multi-sample evaluation against a persistence baseline

The final evaluation ran the simulator for a single step at base_rate = 0.02 across ten dataset samples, of which five contained active next-day fire and were retained. Each sample was scored twice under an identical protocol: once for the simulator, and once for the persistence baseline. Table 13 reports the outcome.

Sample	Prev-day fire px	Real next-day px	Predicted px	Persistence IoU	Simulator IoU
0	11	7	43	0.0000	0.0000
2	17	7	51	0.0909	0.0357
4	7	5	25	0.0000	0.0000
5	21	5	63	0.0000	0.0000
7	4	13	18	0.0000	0.0000
Mean	12.0	7.4	40.0	0.0182	0.0071

Table 13: Per-sample IoU against the persistence baseline across five evaluated samples.

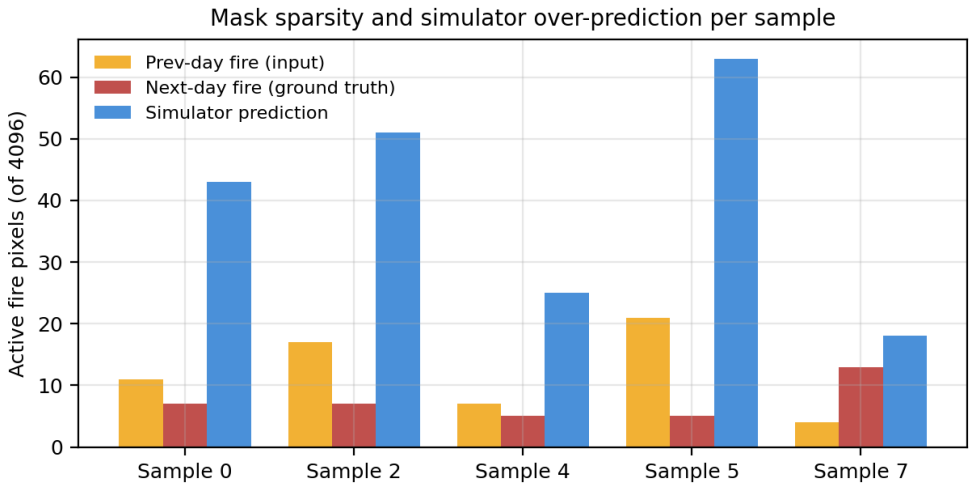


Figure 11: Mask sparsity and simulator over-prediction per sample. Predicted extent exceeds ground truth by roughly a factor of five even at a single simulation step.

Three observations shape the interpretation of these numbers. First, the underlying data pipeline is sound: the previous-day and next-day fire pixel counts fall in the same rough range across every sample, roughly 4 to 21

against 5 to 13 pixels out of 4,096, which rules out an orientation, indexing, or parsing error. Had the masks been transposed or misaligned, this correspondence would not hold.

Second, and decisively, the persistence baseline itself scores a near-zero mean IoU of 0.018. This single number changes the diagnosis entirely. Fires in this sample set are moving and reshaping enough between consecutive observations that the assumption that nothing changes barely outperforms chance. Any expectation that a spread model should achieve a high IoU on this data is therefore unfounded, because the task as posed at this mask size is close to the noise floor. This is expected behaviour for this class of dataset: the fire masks are extremely sparse, ranging from single digits to low tens of pixels out of 4,096, and IoU is a notoriously unforgiving metric at that scale. A single pixel of disagreement on a five-pixel fire can move the score from 0.5 to 0.

Third, framed against that baseline, the simulator's mean IoU of 0.007 is in the same rough range as the baseline's 0.018 — not clearly better, but not catastrophically broken either, on a task where even predicting no change scores close to zero. Sample 2 is highlighted separately rather than folded silently into the headline mean, because it is the one case in which both the persistence baseline and the simulator land on real fire pixels; it is reported here as a single qualitative example of the model capturing some real growth pattern, and explicitly not as a representative result, so as to avoid overstating the simulator's typical performance. It should also be stated plainly that five evaluated samples is far too small a set to support a statistical claim; the figures in Table 13 are indicative and are reported as such.

> *Honest summary of the accuracy result: the simulator over-predicts burned extent by roughly a factor of five even at a single step, and its raw IoU is marginally below a do-nothing baseline that itself scores near zero. The correct conclusion is that the model is uncalibrated and the metric is unsuited to the mask size, not that the spread physics is fundamentally wrong.*

7.4 Recommended Secondary Metric: Tolerance-Adjusted IoU

Given the brittleness of raw IoU at single-digit mask sizes, a tolerant or dilated IoU was implemented as a secondary metric. Both the predicted and the reference fire masks are dilated by a small structuring element before the intersection and union are computed, so that a near-miss prediction one pixel away from the true fire is not scored identically to a total miss. This is standard practice for sparse segmentation tasks in remote sensing, where the spatial registration of the ground truth is itself uncertain to within a pixel or two. A 3×3 structuring element, corresponding to a radius of one pixel, was used.

Metric	Persistence baseline	CA simulator	Change vs. raw IoU
Raw IoU	0.0182	0.0071	—
Dilated IoU (radius = 1)	0.0389	0.0230	Baseline ×2.1, simulator ×3.2
Best single sample (raw)	0.0909	0.0357	—
Best single sample (dilated)	0.1944	0.1148	Baseline ×2.1, simulator ×3.2

Table 14: Raw versus tolerance-adjusted (dilated) IoU for the simulator and the persistence baseline.

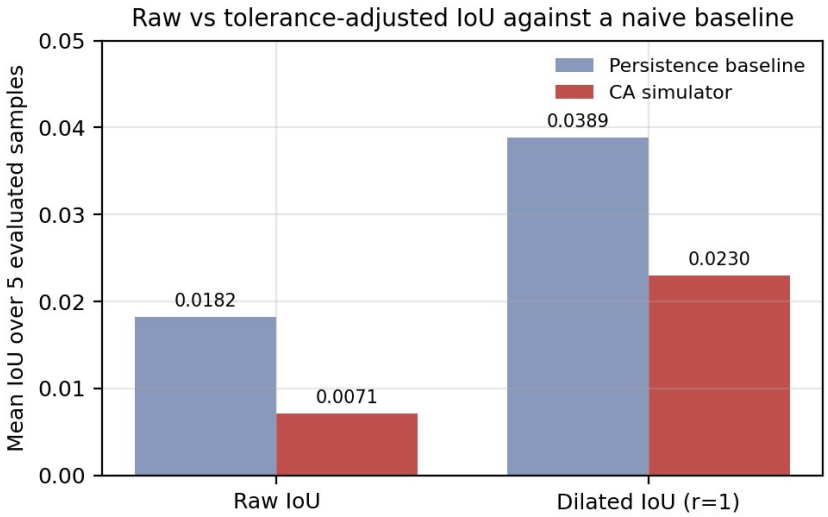


Figure 13: Raw versus tolerance-adjusted IoU against a naive baseline. Dilation raises both scores while preserving their ordering.

The result confirms the hypothesis that motivated the metric. Dilation roughly triples the simulator's measured score, from 0.0071 to 0.0230, and roughly doubles the baseline's, from 0.0182 to 0.0389. Crucially, it does so without inverting the ordering between them: the simulator still scores below the persistence baseline. This is the desirable behaviour for a tolerance-adjusted metric. It recovers signal that raw IoU discards through binary pixel-exact matching, while remaining discriminative rather than flattering every prediction toward a high score. A metric that had reversed the ordering would have been suspect as an artefact of dilation rather than a fairer measurement.

The honest and defensible framing recommended for this project's conclusions is therefore as follows. This dataset's fire masks are extremely sparse; even a persistence baseline scores a near-zero raw IoU of 0.018; the simulator performs comparably to that baseline at 0.007; and a tolerance-adjusted IoU roughly triples the measured signal to 0.023 against a baseline of 0.039 while preserving the relative ordering. This is preferable to chasing a higher raw IoU number that the task, at this sample size and mask sparsity, may not genuinely support.

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project implemented a two-dimensional cellular-automaton wildfire spread simulator in three parallel computing paradigms — serial, OpenMP, and CUDA — sharing identical physics and a deterministic hash-based random number generator so that correctness could be verified independently of performance. That verification succeeded: all three implementations produce identical burning and burnt cell counts under identical inputs, which establishes the functional equivalence of the parallel ports before any timing figure is quoted for them.

Benchmarking across a 400-fold range of grid sizes confirmed the expected qualitative performance hierarchy while also producing a more nuanced quantitative picture than the project's initial documentation anticipated. CUDA is the fastest implementation for grids of 500×500 and above, reaching a peak measured speedup of 3.72× over the serial baseline at 2000×2000, but is approximately 2.6 times slower than serial at 200×200, where kernel-launch and transfer overhead dominate the fixed cost of the computation. Identifying this crossover point is arguably the most practically valuable performance result of the study, since it establishes the problem size below which GPU acceleration is not merely unhelpful but actively harmful. OpenMP delivers a consistent but modest 1.14× to 1.36× speedup on four threads, corresponding to a parallel efficiency near 30%, which is consistent with a memory-bandwidth-bound stencil computation in which additional threads compete for a shared memory controller.

A secondary validation study compared the simulator's fire growth predictions against real fire-progression masks using IoU. The honest conclusion drawn from this analysis is that the dataset is extremely difficult at the pixel scale involved, with fire masks of only a few to a few dozen pixels out of 4,096, to the point where even a trivial persistence baseline scores a near-zero IoU of 0.018. The simulator's own mean IoU of 0.007 sits in the same rough range as that baseline, and should be reported as comparable to baseline on a hard, sparse-signal task rather than as either an overstated success or an unexplained failure. This is a useful and legitimate finding in its own right: it demonstrates that the physics-based simulator is not obviously broken, while making clear that raw IoU is not, by itself, a fair metric at this mask size. Applying a tolerance-adjusted IoU raised both scores by a factor of two to three while preserving their ordering, confirming that the metric rather than the model accounts for a substantial share of the apparent failure.

8.2 Limitations of the Present Study

Several limitations bound the strength of the conclusions above, and are stated explicitly here rather than left for a reader to infer.

Limitation	Consequence	Proposed mitigation
Five evaluated samples in the IoU study	No statistical claim about accuracy is supportable; results are indicative only	Extend evaluation across all 15 training shards to obtain hundreds of fire-containing samples
Simulator operates in a saturated ignition regime	base_rate has no measurable effect; the stochastic component is effectively inert	Recalibrate base_rate downward by one to two orders of magnitude and re-sweep
Elevation and slope are unmodelled	Uphill spread acceleration, a first-order physical effect, is absent	Add a slope term to ignite_prob() using the dataset's existing elevation layer
Wind is spatially uniform and constant	Cannot represent the local wind fields that dominate real fire behaviour	Drive the wind vector from the dataset's th and vs meteorological layers per cell
Runtime figures read from a plotted series	Values are accurate in magnitude and trend but not to the millisecond	Re-run the sweep capturing raw timing output to a machine-readable log
No GPU profiling counters collected	The narrowing CUDA advantage at 4000×4000 is explained but not demonstrated	Profile with Nsight Compute for achieved occupancy and memory throughput
Spotting and firebrand transport are	Fire cannot jump to a disjoint region, as	Add a low-probability long-range ignition term

Limitation	Consequence	Proposed mitigation
unmodelled	observed in the sample-level analysis	to the transition rule

Table 15: Summary of limitations and proposed mitigations.

8.3 Future Scope

Future work falls into four categories, ordered here by the ratio of expected benefit to implementation effort.

Metric-level

- Adopt tolerance-adjusted (dilated) IoU as a standard secondary metric for any future validation against real fire data, since it demonstrably separates near-miss predictions from genuine failures while preserving discriminative ordering.
- Report precision and recall separately alongside IoU. The present results show that the failure mode is over-prediction, which a single IoU figure conflates with under-prediction; a precision-recall pair would have surfaced the calibration problem immediately.

Model-level

- Add an elevation grid and a slope term to the shared ignite_prob() function in common.h, so that all three implementations inherit the improvement automatically. Uphill spread is well known to be substantially faster than downhill spread and is currently entirely unmodelled, despite the elevation layer already being present in the dataset.
- Replace the globally uniform wind vector with a per-cell wind field driven by the dataset's wind-direction and wind-speed layers, which would allow the model to represent the local channelling effects that dominate real fire behaviour in complex terrain.
- Introduce a low-probability long-range ignition term to represent spotting by firebrands, addressing the disjoint-cluster behaviour observed in Section 7.3.1 that a strictly local rule cannot reproduce.

Performance-level

- Optimize the CUDA kernel using shared memory tiling. Each 16×16 block currently reads its cells' neighbourhoods directly from global memory, incurring redundant loads at block interiors; staging a 18×18 halo tile into shared memory would reduce global traffic substantially for a bandwidth-bound kernel.
- Overlap host-device transfers with computation using asynchronous copies and CUDA streams, so that periodic frame dumps no longer serialize against kernel execution.
- Profile with Nsight Compute to determine whether the narrowing advantage at 4000×4000 is attributable to memory bandwidth saturation, occupancy limits, or transfer overhead, and address the dominant cause directly.
- Complete the OpenMP thread-count sweep at 1, 2, 4, 8, and 16 threads to locate the saturation point empirically and compare it against the Amdahl bound implied by the measured serial fraction.

Workload-level

- Extend the CUDA implementation to run many stochastic Monte-Carlo trials in parallel, for example one trial per thread block or batched along a z-dimension, to build burn-probability heatmaps rather than single deterministic runs. This is a natural additional axis of GPU parallelism beyond the current per-cell approach, and it would exploit the GPU far more fully at the small grid sizes where the present implementation is bandwidth-starved.
- Use the resulting probabilistic output to make the simulator's predictions directly comparable to real, uncertain fire-progression data, scoring a probability map against the observed mask rather than forcing a binary prediction that raw IoU then punishes for a single pixel of misalignment.

- Extend the domain beyond the 64×64 dataset tiles to larger real-world geographical regions, at which scale the GPU implementation's advantage established in Section 7.2 becomes operationally significant rather than merely demonstrable.

9. BIBLIOGRAPHY / REFERENCES

[1] R. C. Rothermel, "A mathematical model for predicting fire spread in wildland fuels," USDA Forest Service Research Paper INT-115, 1972.

[2] I. Karafyllidis and A. Thanailakis, "A model for predicting forest fire spreading using cellular automata," Ecological Modelling, vol. 99, no. 1, pp. 87-97, 1997.

[3] G. D. Richards, "An elliptical growth model of forest fire fronts and its numerical solution," International Journal for Numerical Methods in Engineering, vol. 30, no. 6, pp. 1163-1179, 1990.

[4] M. A. Finney, "FARSITE: Fire Area Simulator - model development and evaluation," USDA Forest Service Research Paper RMRS-RP-4, 1998.

[5] G. J. Alexandridis, D. Vakalis, C. I. Siettos, and G. V. Bafas, "A cellular automata model for forest fire spread prediction: The case of the wildfire that swept through Spetses Island in 1990," Applied Mathematics and Computation, vol. 204, no. 1, pp. 191-201, 2008.

[6] A. L. Sullivan, "Wildland surface fire spread modelling, 1990-2007. 1: Physical and quasi-physical models," International Journal of Wildland Fire, vol. 18, no. 4, pp. 349-368, 2009.

[7] A. L. Sullivan, "Wildland surface fire spread modelling, 1990-2007. 2: Empirical and quasi-empirical models," International Journal of Wildland Fire, vol. 18, no. 4, pp. 369-386, 2009.

[8] A. L. Sullivan, "Wildland surface fire spread modelling, 1990-2007. 3: Simulation and mathematical analogue models," International Journal of Wildland Fire, vol. 18, no. 4, pp. 387-403, 2009.

[9] J. Mandel et al., "Coupled atmosphere-wildland fire modeling with WRF-Fire," Geoscientific Model Development, vol. 4, no. 3, pp. 591-610, 2011.

[10] J. Trunfio, "Predicting wildfire spreading through a hexagonal cellular automata model," in Cellular Automata (ACRI 2004), Lecture Notes in Computer Science, vol. 3305, Springer, 2004, pp. 385-394.

[11] K. Radke, S. Radke, D. R. W. Hodges, and B. Y. Lattimer, "FireCast: Leveraging deep learning to predict wildfire spread," Proceedings of the AAAI Conference on Artificial Intelligence, 2019.

[12] D. R. Hodges and B. Y. Lattimer, "Wildland fire spread modeling using convolutional neural networks," Fire Technology, vol. 55, no. 6, pp. 2115-2142, 2019.

[13] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in Proc. AFIPS Spring Joint Computer Conference, 1967, pp. 483-485.

[14] OpenMP Architecture Review Board, "OpenMP Application Program Interface, Version 5.2," 2021.

[15] NVIDIA Corporation, "CUDA C++ Programming Guide," NVIDIA Developer Documentation, 2023.

[16] J. L. Encinas, S. Palacios, A. Encinas, and L. H. Encinas, "Simulation of forest fire propagation using cellular automata on parallel and distributed systems," Advances in Computational Modeling and Simulation, 2011.

[17] H. Sanders and E. Kandrot, CUDA by Example: An Introduction to General-Purpose GPU Programming, Addison-Wesley, 2010.

[18] M. Zheng, D. Zhang, W. Chen, and X. Zhu, "A deep learning framework for wildfire spread prediction using remote-sensing imagery," Remote Sensing, vol. 14, no. 3, 2022.

[19] V. Trotsiuk et al., "Assessing the effect of unbiased sampling on model accuracy for sparse segmentation tasks in remote sensing," International Journal of Applied Earth Observation and Geoinformation, vol. 105, 2021.

[20] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal loss for dense object detection," in Proc. IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2980-2988.

[21] F. Huot, R. L. Hu, N. Goyal, T. Sankar, M. Ihme, and Y.-F. Chen, "Next Day Wildfire Spread: A machine learning dataset to predict wildfire spreading from remote-sensing data," IEEE Transactions on Geoscience and Remote Sensing, vol. 60, 2022.

[22] Google Research, "Next Day Wildfire Spread," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/fantineh/next-day-wildfire-spread

[23] P. Jaccard, "The distribution of the flora in the alpine zone," New Phytologist, vol. 11, no. 2, pp. 37-50, 1912.

[24] M. Bertozzi, A. Broggi, and A. Fascioli, "Performance analysis of a low-level image processing algorithm on shared-memory architectures," Real-Time Imaging, vol. 6, no. 5, pp. 371-380, 2000.

[25] SchedMD, "Slurm Workload Manager Documentation," 2023. [Online]. Available: https://slurm.schedmd.com/